

MERN Deployment Handbook - Phase 1 (Git & GitHub)

These notes match the explanations given during our session. Every command includes: Goal, Concept, Syntax Breakdown, Background Process, Industry Use and Expected Output.

Step 1 - Open the Root Project

Goal: Open the root folder 'bitu website' in VS Code, not only frontend or backend.

Why? Git should manage the complete MERN project. If only frontend is opened, Git cannot manage the complete application.

Correct Structure:

```
bitu website/  
├── backend/  
├── frontend/  
├── .gitignore  
└── README.md
```

Step 2 - VS Code Integrated Terminal

Why use the VS Code terminal?

- One place for Git
- Node.js
- MongoDB
- Render
- Vercel
- NPM

Industry developers mostly work from the integrated terminal.

Step 3 - git status

Command:

git status

Goal: Check the current condition of the repository.

Command Breakdown:

git -> Run the Git program.

status -> Ask Git to display the current repository state.

Background:

Git checks:

- Current Branch
- Modified Files
- Staged Files
- Untracked Files
- Last Commit

Your Output Explained:

On branch master

No commits yet

Untracked files

Meaning:

- You were on master branch.
- No history existed.
- Git could see backend/, frontend/, README.md and .gitignore but was not tracking them.

Step 4 - git add .

Command:

git add .

Goal: Move files from the Working Directory to the Staging Area.

Command Breakdown:

git -> Run Git

add -> Stage files

. -> Current folder and everything inside it

Background:

Working Directory

|

git add .



Staging Area

Important: git add DOES NOT save history. It only prepares files for commit.

LF vs CRLF Warning

Warning received:

LF will be replaced by CRLF

Meaning:

Linux uses LF.

Windows uses CRLF.

This is only a line-ending warning and does not affect deployment.

Embedded Git Repository Problem

Warning:

adding embedded git repository: frontend

Reason:

frontend contained its own .git folder.

Wrong Structure:

```
bitu website/  
├── .git  
└── frontend/  
    ├── .git
```

Correct Structure:

```
bitu website/  
├── .git  
└── frontend/
```

Fix:

Delete ONLY frontend/.git. Never delete the frontend folder itself.

Step 5 - git commit

Command:

```
git commit -m "Initial commit"
```

Goal: Create the first permanent snapshot.

Command Breakdown:

git -> Run Git

commit -> Save a permanent snapshot

-m -> Message

"Initial commit" -> Commit name

Workflow:

Working Directory -> Staging Area -> Local Repository

Step 6 - GitHub Repository

Repository Name: bitu_website

Visibility: Public

README, .gitignore and License were left unchecked because they already existed locally.

Step 7 - Connect Local Git with GitHub

Command:

```
git remote add origin https://github.com/Shreyansh713/bitu_website.git
```

Command Breakdown:

git -> Run Git

remote -> Work with remote repositories

add -> Add a new remote

origin -> Nickname for GitHub repository

URL -> Actual GitHub repository address

This command DOES NOT upload code. It only saves the connection.

Step 8 - Verify Connection

Command:

```
git remote -v
```

Output:

```
origin ... (fetch)
```

```
origin ... (push)
```

fetch = Download changes from GitHub.

push = Upload changes to GitHub.

Step 9 - First Push

Command:

```
git push -u origin master
```

Command Breakdown:

git -> Run Git

push -> Upload commits

-u -> Set upstream (remember this branch)

origin -> GitHub nickname

master -> Branch to upload

Your Output Explained:

Enumerating objects -> Counting files

Compressing objects -> Compressing data

Writing objects -> Uploading data

Resolving deltas -> Comparing changes

master -> master -> Branch created on GitHub

set up to track origin/master -> Future 'git push' is enough

Complete Git Workflow

1. Create/Edit Files
2. `git status`
3. `git add .`
4. `git commit -m "message"`
5. `git push`
6. GitHub updated

What We Will Learn Next

MongoDB Atlas

Environment Variables

Render Deployment

Vercel Deployment

Frontend + Backend Connection

CORS

Production Testing